Data structures and its applications
Unit - I

Compiled by
M S Anand

Department of Computer Science (AI&ML)

Data Structures and its applications

Introduction

Text Book(s):

1. "Data Structures using C / C++", Langsum Yedidyah, Moshe J Augenstein, Aaron M Tenenbaum Pearson Education Inc, 2nd edition, 2015.

Reference Book(s):

1. "Data Structures and Program Design in C", Robert Kruse, Bruce Leung, C.L Tondo, Shashi Mogalla, Pearson, 2nd Edition, 2019

Data Structures and its applications

The syllabus

The syllabus is here.

Data Structures and its applications

Evaluation guidelines

Refer to this.

An overview of C

Primitive data types

.

char

short

integer

long

float

double

Data structures and its applications

An overview of C – Data types and range of values

Variable type	Keyword	Bytes required	Range
Character	char	1	-128 to 127
Unsigned character	unsigned char	1	0 to 255
Integer	int	2 (?)	-32768 to 32767
Short integer	short int	2	-32768 to 32767
Long integer	long int	4	-2,147,483,648 to 2,147,483,647
Unsigned integer	unsigned int	2(?)	0 to 65535
Unsigned short integer	unsigned short	2	0 to 65535
Unsigned long integer	unsigned long	4	0 to 4,294,967,295
Float	float	4	-3.4E+38 to +3.4E+38
Double	double	8	-1.7E+308 to +1.7E+308
Long long	long long	8	
Long double	long double	10	

Basic aritmetic operations:

Addition – '+'

Subtraction – '-'

Multiplication – '*'

Division – '/'

Modulo division – '%'

Unary minus operator
It produces a positive result when applied to a negative operand and a negative result when the operand is positive.

An overview of C

Assignment operators:

=	Example: sum = 10
+=	sum += 10; (Same as sum = sum + 10)
-=	sum -= 10;
*=	sum *= 10;
/=	sum /= 10;
%=	sum %= 10;

……

An overview of C

Relational operators:

>	x > y
<	x < y
>=	x >= y
<=	x <= y
!=	x != y
==	x == y

An overview of C

Increment and decrement operators:

.

Pre and post increment

Pre and post decrement

variable ++

++ variable

variable –

--variable

Logical operators:

&& - Logical AND	True only if all operands are true
|| - Logical OR	True if any of the operands is true.
! – Logical NOT	True if the operand is zero.

An overview of C

Bitwise operators:

&	-	Bitwise AND
|	-	Bitwise OR
^	–	Bitwise exclusive OR
~	-	Bitwise complement
<<	-	Shift left
>>	-	Shift right

Other operators

Comma operators are used to link related expressions together. For example:

```
int a, c = 5, d;
```

sizeof operator

Decision making

.

if, else, else if

switch case

The ternary operator

condition ? expression1 : expression2

```
x = y > 7 ? 25 : 50;
```

The goto statement

```
goto label;
```

label:

Type conversions

Explicit – You do it yourself in the program

Implicit – The compiler does it for you

Enumerations

enum Weekday {Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday};

enum Weekday today = Wednesday;

An overview of C

Operator precedence and associativity
The C operator precedence table is here.

The loops:

The “**for**” loop

The “**while**” loop

The “**do … while**” loop

The **break** statement

The **continue** statement

Arrays

Index starts from 0

Single, Multi-dimensional arrays

Row major or Column major ?

A string in C is a character array terminated with '\0'

An overview of C

The nine most commonly used functions in the string library are:
strcat - concatenate two strings
strchr - string scanning operation
strcmp - compare two strings
strcpy - copy a string
strlen - get string length
strncat - concatenate one string with part of another
strncmp - compare parts of two strings
strncpy - copy part of a string
strrchr - a pointer to the **last** occurrence of c within s instead of the first.

Using the sizeof operator with arrays

The sizeof operator executed on an array gives the size of the array in bytes:

```
char arr [20];
sizeof (arr) – results in 20 bytes
```

What about the following:

```
int iarr [20];
sizeof (iarr)   -  ??

long larr [40];
sizeof (larr)   -   ??
```

```
const char hex_array[] = {‘A’, ‘B’};
```

Pointers

The pointer in C language is a variable which stores the address of another variable. This variable can be of type int, char, array, function, or any other pointer.

Arrays and Pointers – Differences

1. the sizeof operator
   a. sizeof(array) returns the amount of memory used by all elements in array
   b. sizeof(pointer) only returns the amount of memory used by the pointer variable itself.
2. the & operator
   a. &array is an alias for &array[0] and returns the address of the first element in array
   b. &pointer returns the address of pointer
3. a string literal initialization of a character array
   a. char array[] = “abc” sets the first four elements in array to ‘a’, ‘b’, ‘c’, and ‘\0’
   b. char *pointer = “abc” sets pointer to the address of the “abc” string (which may be stored in read-only memory and thus unchangeable)
4. Pointer variable can be assigned a value whereas array variable cannot be.
5. Arithmetic on pointer variable is allowed.

Pointers
Note

char* is a **mutable** pointer to a **mutable** character/string.

const char* is a **mutable** pointer to an **immutable** character/string. You cannot change the contents of the location(s) this pointer points to. Also, compilers are required to give error messages when you try to do so. For the same reason, conversion from const char * to char* is deprecated.

char* const is an **immutable** pointer (it cannot point to any other location) **but** the contents of location at which it points are **mutable**.

const char* const is an **immutable** pointer to an **immutable** character/string.

void *ptr;

Structure

A structure is a user defined data type in C.

A structure creates a data type that can be used to group items of possibly different types into a single type.

How to create a structure?

'struct' keyword is used to create a structure.

```
struct address
{
   char name[50];
   char street[100];
   char city[50];
   char state[20];
   int pin;
};
```

An overview of C

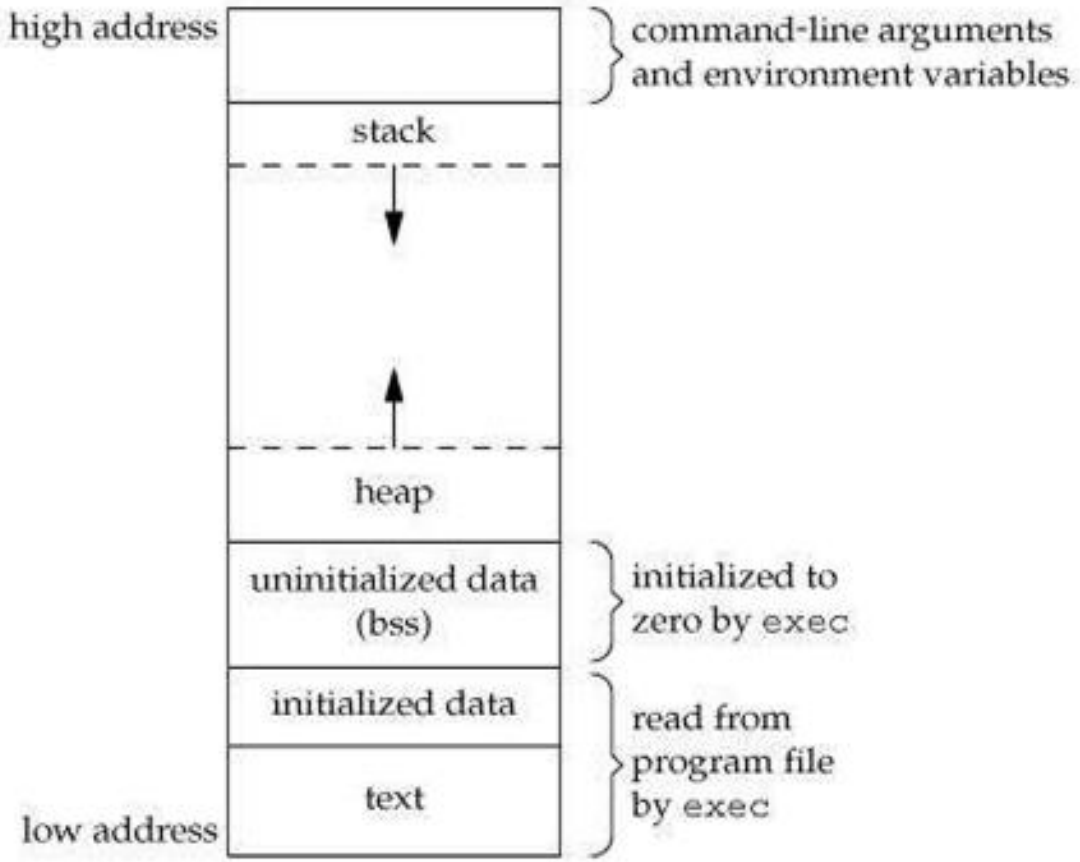
Accessing structure members through the “.” Operator.

.

If you have a pointer to a structure, then use the -> operator.

sizeof a structure.

Memory layout of a C program



Dynamic memory allocation

void *malloc(int size);

void *calloc(int nmemb, int size); – Memory set to zero.

void *realloc(void *ptr, size_t size);

free(void *ptr);

Dangling pointer and memory leak

- A dangling pointer points to a memory address that has already been freed or deallocated, leading to crashes or bad data if used.
- A memory leak happens when allocated dynamic memory is never freed, but the program loses the pointer to it, making that memory impossible to reuse.

Bit fields

```
// A space optimized representation of date
struct date
{
   // d has value between 1 and 31, so 5 bits are sufficient
   unsigned int d: 5;

   // m has value between 1 and 12, so 4 bits are sufficient
   unsigned int m: 4;

   unsigned int y;
};
```

Unions

User defined data type just like a structure.

```
union sample
{
    char name [4];
    long  length;
    short s1;
};
```

An overview of C

```
#include <stdio.h>

int main (void)
{
    union long_bytes
            {
                char ind [4];
                long ll;
            } u1;

            long l1 = 0x10203040;
            int i;
            u1.ll = l1;

            for (i = 0; i < sizeof (long); i ++)
                        printf ("Byte %d is %x\n", i, u1.ind [i]);
            return 0;
}
```

Little Endian and Big Endian

C language uses 4 storage classes,
auto, **extern**, **static** and **register**.

auto
This is the default storage class for all the variables declared **inside a function or a block**. Hence, the keyword auto is rarely used while writing programs in C language.

extern:
Extern storage class simply tells us that the variable is defined elsewhere and not within the same block where it is used.

static
Their scope is local to the function to which they were defined.

register

This storage class declares register variables which have the same functionality as that of the auto variables. The only difference is that the compiler tries to store these variables in the register of the microprocessor if a free register is available.

volatile

C's volatile keyword is a qualifier that is applied to a variable when it is declared. It tells the compiler that the value of the variable may change at any time--without any action being taken by the code the compiler finds nearby.

Functions in C

Function declaration

Function call

Function definition

Pointers to functions

function_return_type(*Pointer_name)(function argument list)

For example:

double (*p2f)(double, char)

Command line arguments

```
int main (int argc, char **argv)
{
}
```

argc ??

argv[0] - ??

How to pass an argument like “PES University”?

Environment variables

int main (int argc, char **argv, char **envp)

extern char **environ;

Functions with variable number of arguments

```
int func(int, ... )
{ . . .
}

int main()
{
        func(1, 2, 3);
        func(1, 2, 3, 4);
}
```

Data structures and its applications

Keywords in C

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

Data structures and its applications

Introduction

What is a data structure?

"A data structure is a specialized format for organizing, processing, retrieving and storing data".

"A data structure is a way of **organizing** and **storing** data in a computer so that it can be **accessed and used efficiently**. It refers to the logical or mathematical representation of data, as well as the implementation in a computer program".

An Abstract Data Type (ADT) defines data and operations but, no implementation.

Example of an ADT – List

Store a given number of elements of any type

Read elements by position

Modify element at a position

Example of implementation of a list – Array, Linked List.

Data structures and its applications

Introduction ..

There are several basic and advanced types of data structures, all designed to arrange data to suit a specific purpose.

Data structures make it easy for users to access and work with the data they need.

Most importantly, data structures frame the organization of information so that machines and humans can better understand it.

In computer science and computer programming, a data structure might be selected or designed to store data for the purpose of using it with various algorithms -- commonly referred to as data structures and algorithms (DSA).

Introduction ..

In some cases, the algorithm's basic operations are tightly coupled to the data structure's design.

What does a data structure contain?
Each data structure contains information about the data values; relationships between the data; and, in some cases, functions that can be applied to the data.

Data structures and its applications

Introduction …

Why are data structures important?
Typical base data types, such as integers or floating-point values, that are available in most computer programming languages are generally insufficient to capture the logical intent for data processing and use.

Yet applications that ingest, manipulate and produce information must understand how data should be organized to simplify processing.

Data structures bring together the data elements in a logical way and facilitate the effective use, persistence and sharing of data.

They provide a formal model that describes the way the data elements are organized.

Data structures and its applications

Introduction …

Data structures are the building blocks for more sophisticated applications.

They're designed by composing data elements into a logical unit representing an abstract data type that has relevance to the algorithm or application. An example of an abstract data type is a customer name that's composed of the character strings for first name, middle name and last name.

It's not only important to use data structures, but it's also important to choose the proper data structure for each task. Choosing an ill-suited data structure could result in slow runtimes or unresponsive code.

Data structures and its applications

Introduction …

Examples of how data structures are used include the following:

Storing data. Data structures are used for efficient data persistence, such as specifying the collection of attributes and corresponding structures used to store records in a database management system.

Managing resources and services. Core operating system resources and services are enabled using data structures such as linked lists for memory allocation, file directory management and file structure trees, as well as process scheduling queues.

Data exchange. Data structures define the organization of information shared between applications, such as TCP/IP packets.

Data structures and its applications

Introduction …

Ordering and sorting. Data structures such as binary search trees -- also known as an *ordered or sorted binary tree* -- provide efficient methods of sorting objects, such as character strings used as tags. With data structures such as priority queues, programmers can manage items organized according to a specific priority.

Indexing. Even more sophisticated data structures such as B-trees are used to index objects, such as those stored in a database.

Searching. Indexes created using binary search trees, B-trees or hash tables speed the ability to find a specific sought-after item.

Scalability. Big data applications use data structures for allocating and managing data storage across distributed storage locations, ensuring scalability and performance.

Data structures and its applications

Introduction …

Types of data structures

Linear

Data elements in a linear data structure are linked to one another in a sequential arrangement, with each element linked to the elements in front of and behind it. In this manner, a single run can traverse the structure.

Non-linear

The data structure in which the data elements are randomly arranged. The elements are not arranged sequentially. The data elements are present at different levels. In Non-linear data structures, there are different paths for an element to reach the other element. The data elements in the non-linear data structures are connected to one or more elements.

Data structures and its applications

Data structures most commonly used

Lists

- Singly linked
- Doubly linked
- Circularly linked
- Multilist

Stack

Queues

Trees

Graphs

List

List as an ADT

A static list

Store a given number of elements of a given data type
Write/Modify element at a position
Read element at a position

Size is predefined and static
Possible implementation – using Arrays -> ArrayList

A sample implementation is here MainProg.c ArrayList.c

Data structures and its applications

Linked list

A dynamic list

- Empty list (size is 0)
- Insert an element at a given position
- Remove an element from a given position
- Count the number of elements
- Read/Modify any element
- Specify data type for the element

Declare a very big array and manipulate the elements as required.
Deciding the maximum size is tough.
Access is sequential and memory allocation is also sequential.

Data structures and its applications

Linked list

Better way of implementing a dynamic list is by using **Linked List**

A Linked list is a linear collection of data elements, whose order is not given by their physical placement in memory. Instead, each element points to the next. It is a data structure consisting of a collection of nodes which together represent a sequence.

Access – Linear, Placement - Random

In its most basic form, each node contains: data, and a reference (in other words, a *link*) to the next node in the sequence.

This structure allows for efficient insertion or removal of elements from any position in the sequence during iteration.

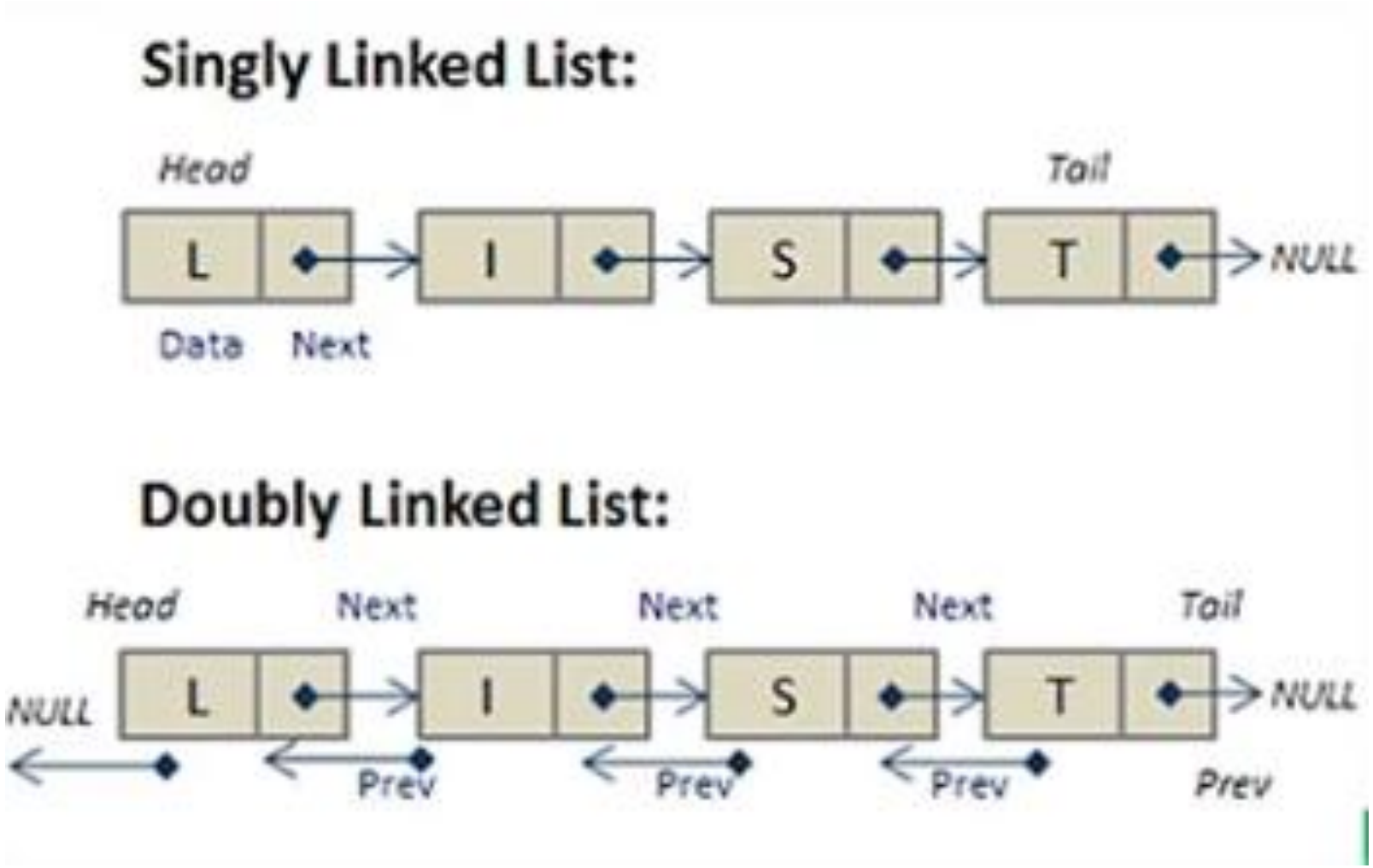
Data structures and its applications

Array List v/s Linked list

Array List	Linked List
It is of fixed size; resizing is expensive	Size is dynamic
Manipulations (insertions and deletions) are expensive	Insertions and Deletions are efficient
Elements are in contiguous locations	Elements are in non-contiguous locations
We might end up wasting most part of the array	Memory is used efficiently
Access (sequential and random) is faster	Access is slower

Data structures and its applications

Linked list ..



Data structures and its applications

Linked list …

The principal benefit of a linked list over a conventional array is that the list elements can be easily inserted or removed without reallocation or reorganization of the entire structure because the data items need not be stored contiguously in memory or on disk, while restructuring an array at run-time is a much more expensive operation. Linked lists allow insertion and removal of nodes at any point in the list, and allow doing so with a constant number of operations by keeping the link previous to the link being added or removed in memory during list traversal.

On the other hand, since simple linked lists by themselves do not allow random access to the data or any form of efficient indexing, many basic operations—such as obtaining the last node of the list, finding a node that contains a given datum, or locating the place where a new node should be inserted—may require iterating through most or all of the list elements.

Data structures and its applications

Linked list …

Linked lists are dynamic, so the length of a list can increase or decrease as necessary. Each node does not necessarily follow the previous one physically in the memory.

Disadvantages

1. They use more memory than arrays because of the storage used by their pointers.
2. Nodes in a linked list must be read in order from the beginning as linked lists are inherently sequential access.
3. Nodes are stored in locations which are not contiguous, greatly increasing the time periods required to access individual elements within the list, especially with a CPU cache.
4. Difficulties arise in linked lists when it comes to reverse traversing. For instance, singly linked lists are cumbersome to navigate backwards and while doubly linked lists are somewhat easier to read, memory is consumed in allocating space for a back-pointer.

Linked list

How do we represent a linked list in C?

.

```
struct abc
{
    int data;
    struct abc *next;
};
```

Creation of a linked list

Traversal through a linked list

Create a linked list from a given array (of structures)

Data structures and its applications

Linked list

Some operations on a linked list

1. Creation of a linked list
2. Linked List Insertion
3. Linked List Deletion (Deleting a given key)
4. Linked List Deletion (Deleting a key at given position)
5. Write a function to delete a Linked List
6. Find Length of a Linked List
7. Search an element in a Linked List
8. Write a function to get N^{th} node in a Linked List
9. N^{th} node from the end of a Linked List
10. Print the middle of a given linked list
11. Write a function that counts the number of times a given int occurs in a Linked List
12. Function to check if a singly linked list is palindrome ??
13. Reverse a linked list

A sample implementation is:

MainProgSLL_2.c SLList_2.c SLList_2.h

Data structures and its applications

Stack

What is a stack?

A stack is a container of objects that are inserted and removed according to the last-in first-out (LIFO) principle.

In the pushdown stacks, only two operations are allowed:

push the item into the stack, and **pop** the item out of the stack.

A stack is a limited access data structure - elements can be added and removed from the stack only at the top.

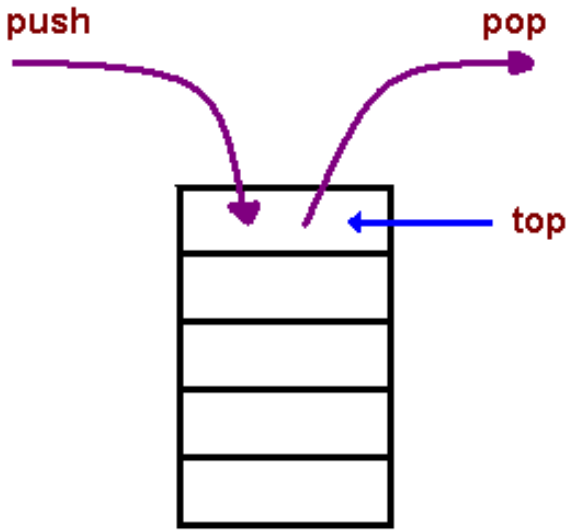
push adds an item to the top of the stack,
pop removes the item from the top.

Data structures and its applications

Stack

A helpful analogy is to think of a stack of books; you can remove only the top book, also you can add a new book on the top.

A stack is a **recursive** data structure (How?)
Here is a structural definition of a Stack:
a stack is either empty or
it consists of a top and the rest which is a stack;



Data structures and its applications

Stack

Applications

The simplest application of a stack is to reverse a word. You push a given word to stack - letter by letter - and then pop letters from the stack.

Another application is an "undo" mechanism in text editors; this operation is accomplished by keeping all text changes in a stack.

Language processing:

space for parameters and local variables is created internally using a stack.

compiler's syntax check for matching braces is implemented by using stack.

support for recursion

Data structures and its applications

Stack

Stack is a linear data structure which follows a particular order in which the operations are performed. The order may be LIFO(Last In First Out) or FILO(First In Last Out).

Mainly the following basic operations are performed in the stack:
Push: Adds an item in the stack. If the stack is full, then it is said to be an Overflow condition.

Pop: Removes an item from the stack. The items are popped in the reversed order in which they are pushed. If the stack is empty, then it is said to be an Underflow condition.

Peek or Top: Returns top element of stack.

isEmpty: Returns true if stack is empty, else false.

isFull - ??

Data structures and its applications

Stack

Implementation:
There are two ways to implement a stack:

- Using arrays

A sample implementation is here:
StackMain_2.c StackOps_2.c StackOps_2.h

- Using linked lists

A sample implementation is here:
MainProgStack.c StackList.c StackList.h

Stack

Applications - functions

Activation record :

When functions are called, the system (or the program) must remember the place where the call was made, so that it can return there after the function is complete.

It must also remember all the local variables, processor registers, and the like, so that information will not be lost while the function is working.

This information is considered as large data structure . This structure is sometimes called the invocation record or the activation record for the function call.

Data structures and its applications

Stack

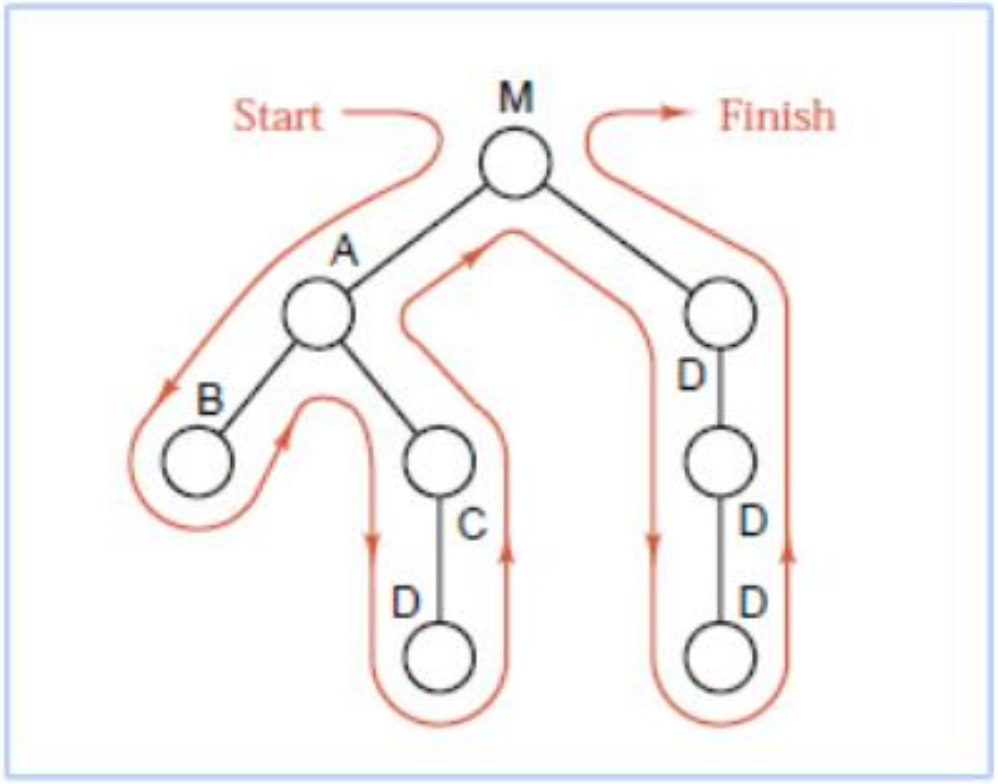
Suppose, there are three functions called A, B and C. M is the main function.
Suppose A invokes B and B invokes C. Then B will not have finished its work until C has finished and returned. Similarly, A is the first to start work, but it is the last to be finished, not until sometime after B has finished and returned.
Thus, the sequence by which function activity proceeds is summed up as the property last in, first out.
The machine's task of assigning temporary storage area (activation records) used by functions would be in same order (LIFO).
Since LIFO property is used, the machine allocates these records in the stack
Hence a stack plays a key role in invoking functions in a computer system.

Data structures and its applications

Stack – Functions, nested functions

Example :

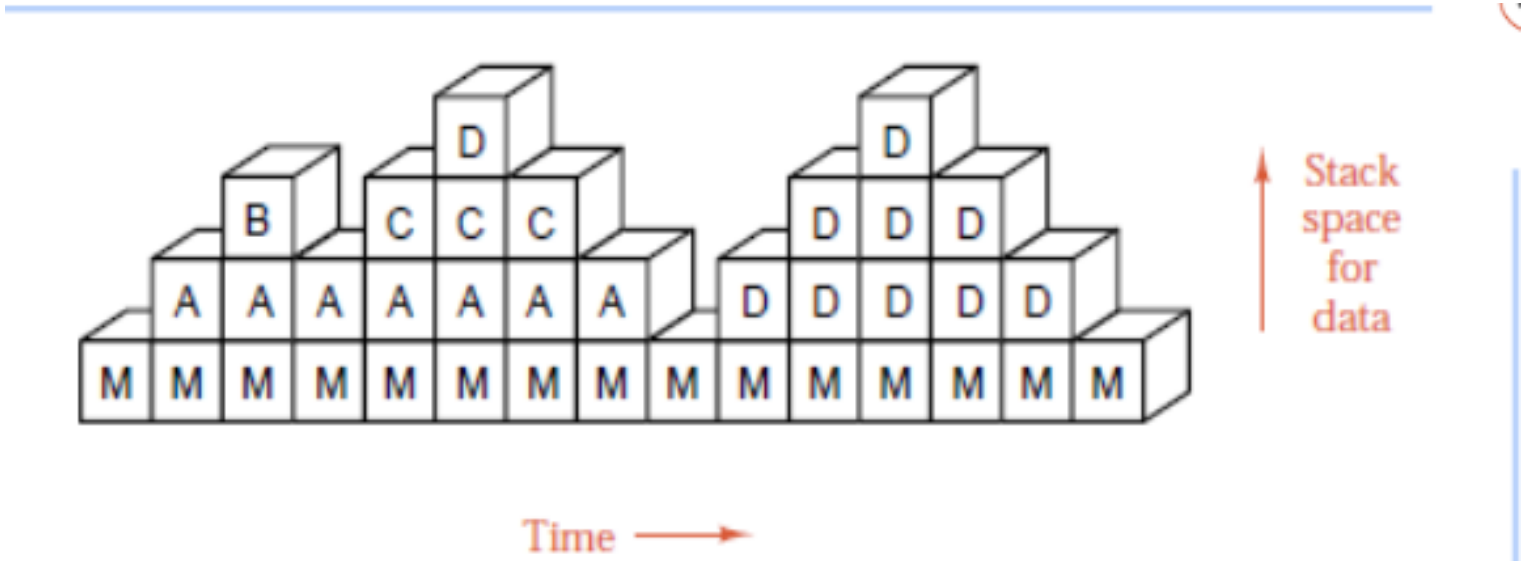
Consider the following showing the order in which the functions are invoked



Data structures and its applications

Stack – Functions, nested functions

The Sequence of stack frames of the activation records of the function calls is given below. Each vertical column shows the contents of the stack at a given time, and changes to the stack are portrayed by reading through the frames from left to right.



How a particular problem is solved using recursion

The idea is to represent a problem in terms of one or more smaller problems.

A way is found to solve these smaller problems and then build up a solution to the entire problem.

The sub problems are in turn broken down into smaller sub problems until the solution to the smallest sub problem is known.

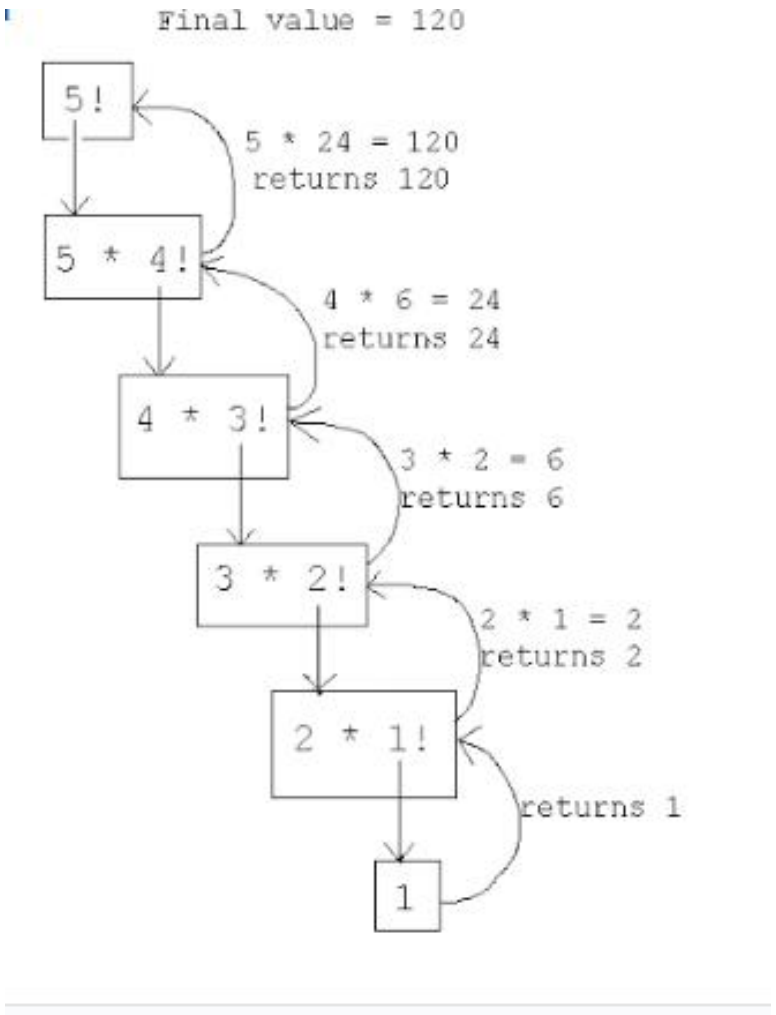
The solution to the smallest sub problem is called the base case.

In the recursive program, the solution to the base case is provided

Data structures and its applications

Stack – Recursion

Example 1: Factorial of a Number n



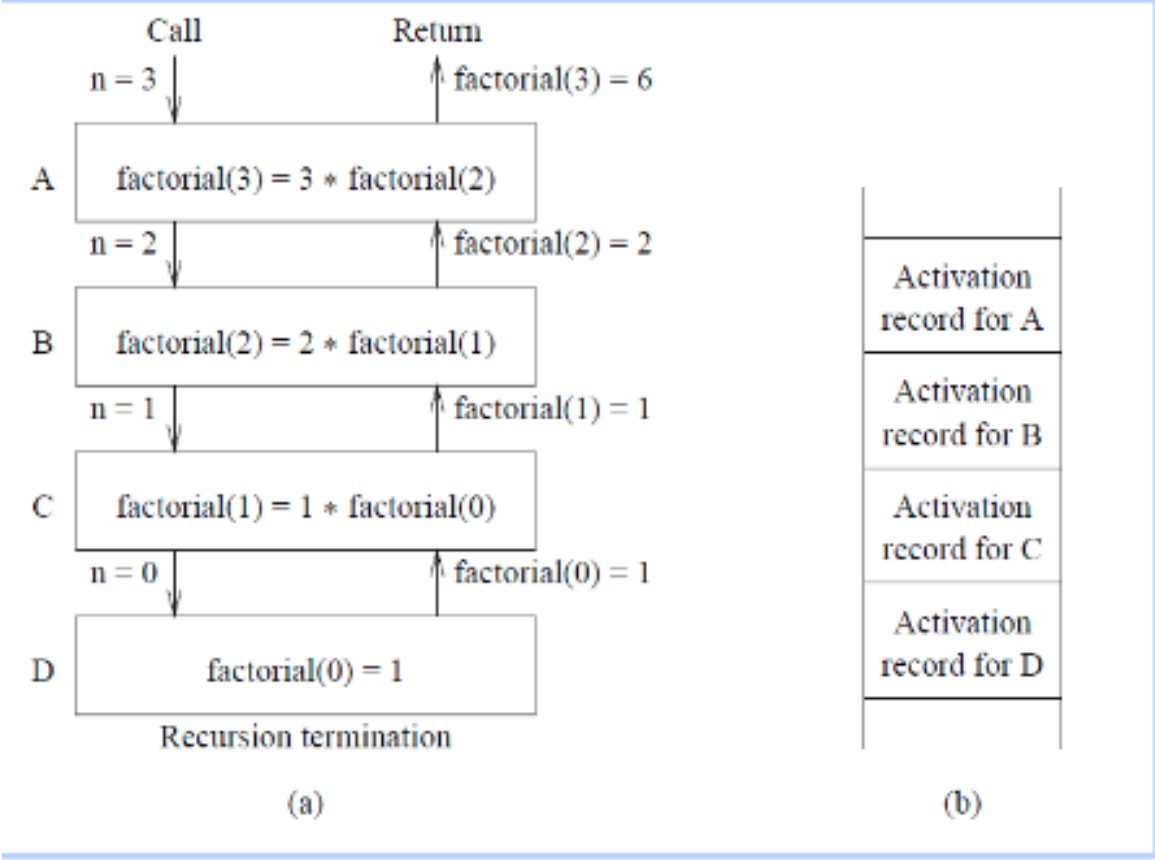
Recursive definition :
n! = 1 if n=1
n!= n * (n-1)! If n>1

Data structures and its applications

Stack – Recursion

Recursive function to compute factorial of n

```
factorial(n)
{
  int f;
  if(n==0)
       return 1;
   f=n*factorial(n-1);
return f;
}
```

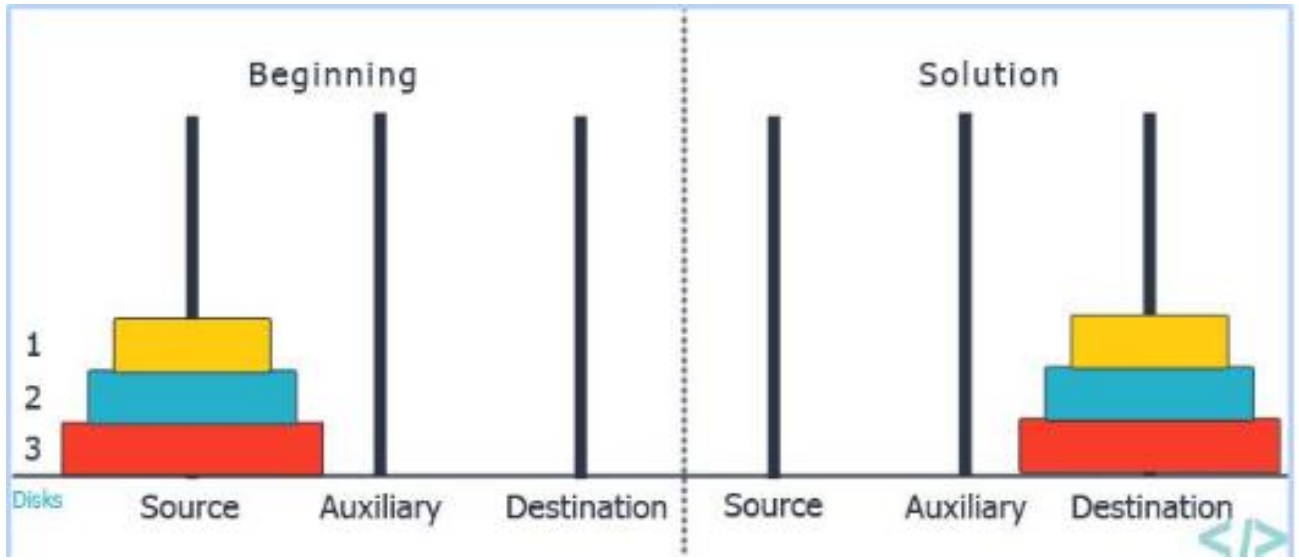


Data structures and its applications

Stack – Tower of Hanoi problem

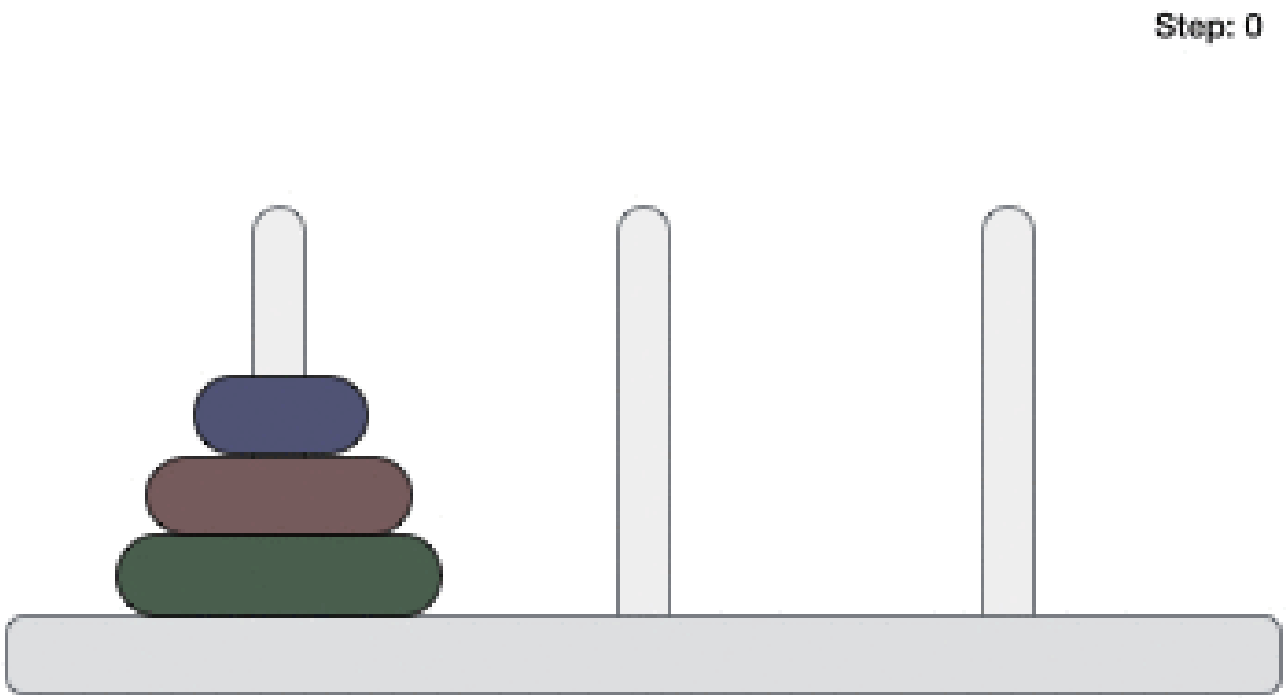
Tower of Hanoi – 3 disks

Three Pegs A, B and C exist. N disks of differing diameters are placed on peg A. The Larger disk is always below a smaller disk. The objective is to move the N disks from Peg A to Peg C using Peg B as the auxillary peg.



Data structures and its applications

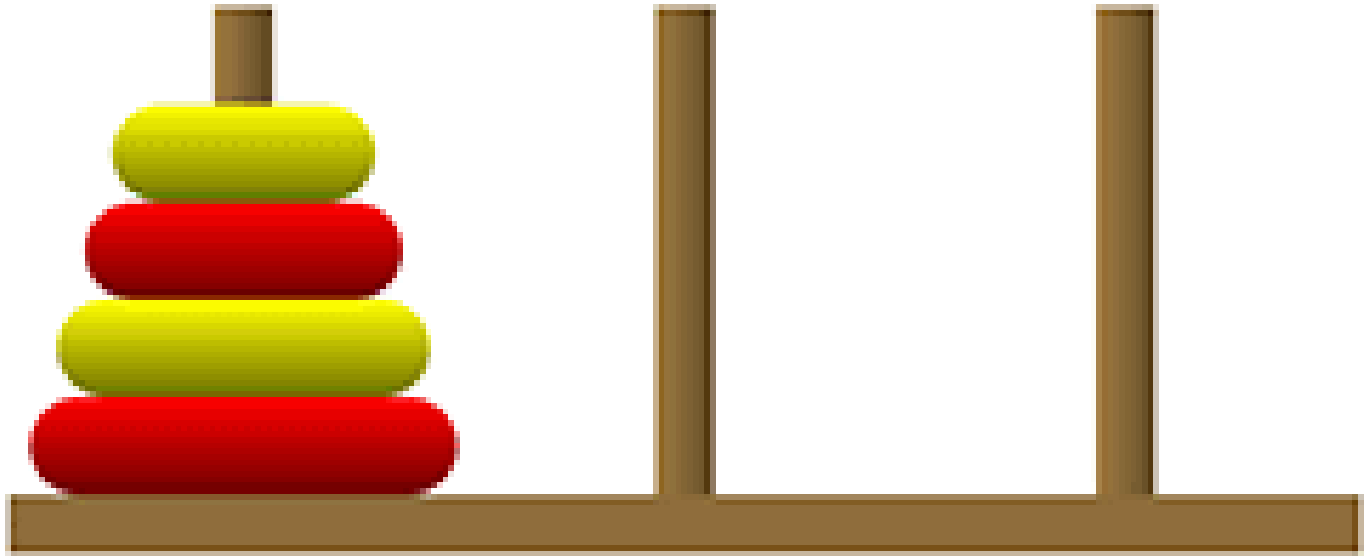
Stack – Tower of Hanoi



Data structures and its applications

Stack – Tower of Hanoi

With 4 disks



Data structures and its applications

Stack – Tower of Hanoi problem

Tower of Hanoi – recursive solution

If a solution to n-1 disks is found, then the problem would be solved. Because in the trivial case of one disk, the solution would be to move the single disk from Peg A to Peg C.

To move n disks from A to C , the recursive solution would be as follows
If n=1 move the single disk from A to C and stop
Move the top n-1 disks from A to B using C as auxillary
Move the remaining disk from A to C
Move n-1 disks from B to C using A as the auxillary

Data structures and its applications

Stack – Tower of Hanoi problem – solution

Recursive function for Tower of Hanoi

```
void tower (int n, char src, char tmp, char dst)
{
        if (n==1)
        {
                printf ("\nMove disk %d from %c to %c",n,src,dst);
                return;
        }
        tower (n-1, src, dst, tmp);
        printf ("\nMove disk %d form %c to %c",n,src,dst);
        tower (n-1, tmp, src, dst);
        return;
}
```

Data structures and its applications

Stack – Tower of Hanoi problem – solution

Recursive function for Tower of Hanoi

Solution for 4 disks

Move disk 1 from peg A to peg B
Move disk 2 from peg A to peg C
Move disk 1 from peg B to peg C
Move disk 3 from peg A to peg B
Move disk 1 from peg C to peg A
Move disk 2 from peg C to peg B
Move disk 1 from peg A to peg B

Solution for moving 3 disks from A to B

Move disk 4 from peg A to peg C

Move 4^{th} disk from A to C

Move disk 1 from peg B to peg C
Move disk 2 from peg B to peg A
Move disk 1 from peg C to peg A
Move disk 3 from peg B to peg C
Move disk 1 from peg A to peg B
Move disk 2 from peg A to peg C
Move disk 1 from peg B to peg C

Solution for moving 3 disks from B to C

Data structures and its applications

Infix, postfix and prefix expressions

Consider the sum of A and B expressed as A + B
This representation is called **infix**.

There are two alternate notations for expressing sum of A and B using the symbols A , B and + , these are

Prefix : + A B
Postfix : A B +

The prefixes “Pre” , “post” and “in” refers to the relative position of the operator with respect to the two operands.

In prefix, operator precedes the two operands

In postfix, the operator follows the two operands

In infix, the operator is in between the two operands

Data structures and its applications

Infix, postfix and prefix expressions

Conversion of Infix to Postfix

- For Example : consider the expression A + B * C
- The evaluation of the above expression requires the knowledge of precedence of operators
- A + B * C can be expressed as A + (B * C) as multiplication takes precedence over addition
- Applying the rules of precedence the above infix expression can be converted to postfix as follows

A + B * C = A + (B * C)

= A + (BC *) convert the multiplication

= A (B C *) + convert the addition

= A B C * +

Data structures and its applications

Infix, postfix and prefix expressions

Conversion of Infix to Prefix

- For Example : consider the expression A + B * C
- Applying the rules of precedence the above infix expression can be converted to prefix as follows

A + B * C = A + (B * C)

= A + (* B C) convert the multiplication

= + A (* B C) + convert the addition

= + A * B C

Data structures and its applications

Infix, postfix and prefix expressions

Applying the rules of precedence the table shows the conversion of Infix to Postfix and Prefix Expression

Infix	Postfix	Prefix
A + B * C + D	A B C * + D +	+ + A * B C D
(A + B) * (C + D)	A B + C D + *	* + A B + C D
A * B + C * D	A B * C D * +	+ * A B * C D
A + B + C + D	A B + C + D +	+ + + A B C D
A $ B * C – D + E / F / (G+H)	A B $ C * D – E F / G H + / +	+ - * $ A B C D / / E F + G H
((A + B)*C – (D – E)) $ (F + G)	A B + C * D E – - F G + $	$ - * + A B C – D E + F G
A – B / (C * D $ E)	A B C D E $ * / -	- A / B * C $ D E

$ is the exponentiation operator and its precedence is from right to left

For example, A $ B $ C = A $ (B $ C)

Data structures and its applications

Infix to postfix conversion algorithm

Algorithm to Convert Infix to Postfix Expression Using Stack

The Infix Notation is traversed from left to right and a stack is used here to maintain the operator to be inserted in the postfix expression.

Watch: https://www.youtube.com/watch?v=7SBN3WCsl3g

1. If we encounter an opening parenthesis (, we push it into the stack.

2. If we have a closing parenthesis), then keep popping out elements from the top of the stack and append them to the postfix expression until the top of the stack contains (. In the end, just pop out the opening parenthesis but make sure to not append this in the postfix expression.

3. If we encounter an operand, then append it to the postfix expression.

Data structures and its applications

Infix to postfix conversion algorithm

4. If we encounter any operator, there exists several cases,
   1. If the top of the stack contains the operator with less precedence in comparison to the operator which is going to be pushed there or stack is empty, then just push the new operator in the stack.
   2. If the top of the stack contains the operator with high or equal precedence, then pop the operators from the stack and append to the postfix expression until the top of the stack does not contain the operator with lower precedence.
   3. The inference of these two points is, that the operator which is at the upper level of the stack must have strictly higher precedence than the one which is at a lower level.

5. In the end if we have nothing to traverse in the Infix Notation; then just append all operators from the stack in the sequence of pop operations.

Data structures and its applications

Infix to postfix conversion algorithm

```
opstk is the empty stack
while(not end of input)
{
        symb = next input character
        // if the input symbol is an operand , add it to the postfix string
         if(symb is an operand)
                 add symb to postfix string
        else
        {
                // pop the contents of the stack while the precedence of
                // top of the stack is greater than the precedence of the
 symbol scanned
                 //prcd function returns true in this case
                while(!empty(opstk ) and ( prcd(stacktop(opstk),symb))
                 {
                         topsymb=pop(opstk)
                        add topsymb to postfix string
                }
```

Data structures and its applications

Infix to postfix conversion algorithm

```
            /* if prcd function returns false, then
            if  the stack is empty and symbol is not ‘)’, push symb on to the
stack*/
             if( empty(opstk) || symb!=‘)’)
                        push(opstk,symb)
            else
                         topsymb=pop(opstk); // if symb is ‘)’ ,  pop ‘ ( ‘ from the
stack
            }
            while(!empty(opstk)
            {
                        topsymb=pop(opstk)
                        add topsymb to postfix string
            }
}
```

Data structures and its applications

Infix to postfix conversion algorithm

•prcd(OP1,OP2) is a function that compares the precedence of the top of the stack(OP1) and the input symbol (OP2) and returns TRUE if the precedence is greater else returns FALSE.

•Some of the return values of prcd function

- prcd(‘ * ‘ , ‘ + ‘) returns TRUE : prcd(‘ * ’, ’ / ’) returns TRUE
- prcd(‘ + ‘, ‘ * ‘) returns FALSE : prcd(‘ / ’, ’ * ’) returns TRUE
- prcd(‘ + ‘ , ‘ + ‘) returns TRUE :
- prcd(‘ + ‘ , ‘ – ‘) returns TRUE
- prcd(‘ – ‘ , ‘ – ‘) returns TRUE
- prcd(‘ $ ‘ , ‘$’) returns FALSE : exponentiation operator, associatively from right to left
- prcd(‘ (‘, op) , prcd(op ‘ (‘) returns FALSE for any operator
- prcd (op , ‘)‘) returns TRUE for any operator
- prcd(‘ (‘ , ‘) ‘) returns FALSE

Data structures and its applications

Infix to postfix conversion algorithm

•The motivation behind the conversion algorithm is the desire to output the operators in the order in which they are to be executed.

•If an incoming operator is of greater precedence than the one on top of the stack, this new operator is pushed on to the stack.

•If on the other hand , the precedence of the new operator is less than the one on the top of the stack, the operator on the top of the stack should be executed first.

•Therefore, the top of the stack is popped out and added to the postfix and the incoming symbol is compared with the new top and so on.

•Parenthesis in the input string override the order of operations

•When a left parenthesis is scanned, it is pushed on to the stack

•When its associated right parenthesis is found, all the operators between the two parenthesis are placed on the output string. Because they are to be executed before any operators appearing after the parenthesis

Data structures and its applications

Infix to prefix conversion algorithm

Example 1:
Input : A * B + C / D
Output : + * A B/ C D

Example 2 :
Input : (A - B/C) * (D/K-L)
Output : *-A/BC-/DKL

1: Reverse the infix expression i.e A+B*C will become C*B+A.
Note while reversing each '(' will become ')' and each ')' becomes '('.

2: Obtain the postfix expression of the modified expression i.e CB*A+.

3: Reverse the postfix expression. Hence in our example prefix is +A*BC.

Data structures and its applications

Linked list – Multi-list

Multi level list

A multilist is a structure in which a number of lists are combined to form a single aggregate structure.

A multi-list is a data structure in which there are multiple links from a node. In a general multi-list, each node can have any number of pointers to other nodes.

Key Characteristics

Multiple Pointers: Each node holds data plus N pointer fields pointing to different next or previous nodes.

Shared Data: The actual data item stays in one memory location while multiple sequence chains thread through it.

Generalization: A doubly linked list is a simple multilist with two opposing pointers.

Data structures and its applications

Linked list – Multi-list

Common Uses

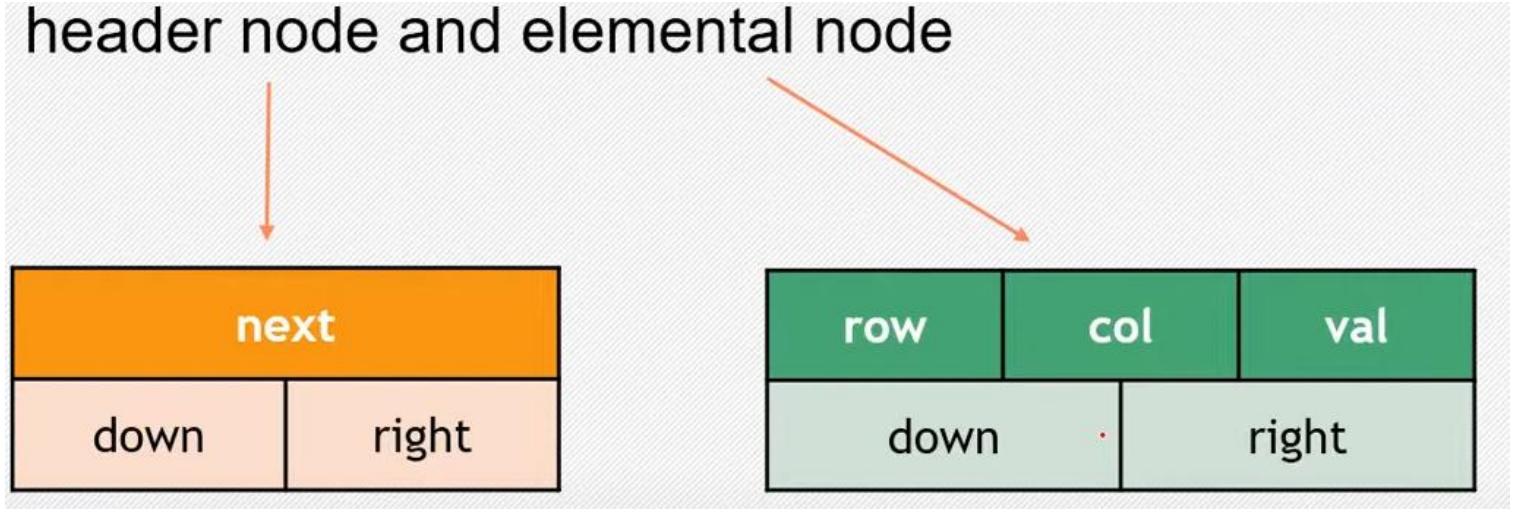
Sorting Multiple Ways: Keeping a student record list threaded alphabetically by name and separately sorted by grade.
Sparse Matrices: Representing large grids with mostly zero values by linking rows and columns together.
Complex Networks: Modeling hierarchical relationships or multi-key database indices.

What is a sparse matrix?

A sparse matrix is a special case of a matrix in which the number of zero elements is much higher than the number of non-zero elements. As a rule of thumb, if 2/3 of the total elements in a matrix are zeros, it can be called a sparse matrix. Using a sparse matrix representation — where only the non-zero values are stored — the space used for representing data and the time for scanning the matrix are reduced significantly.

Linked list – Multi-list

Sparse matrix node structure



Header node has three pointers – next, down and right

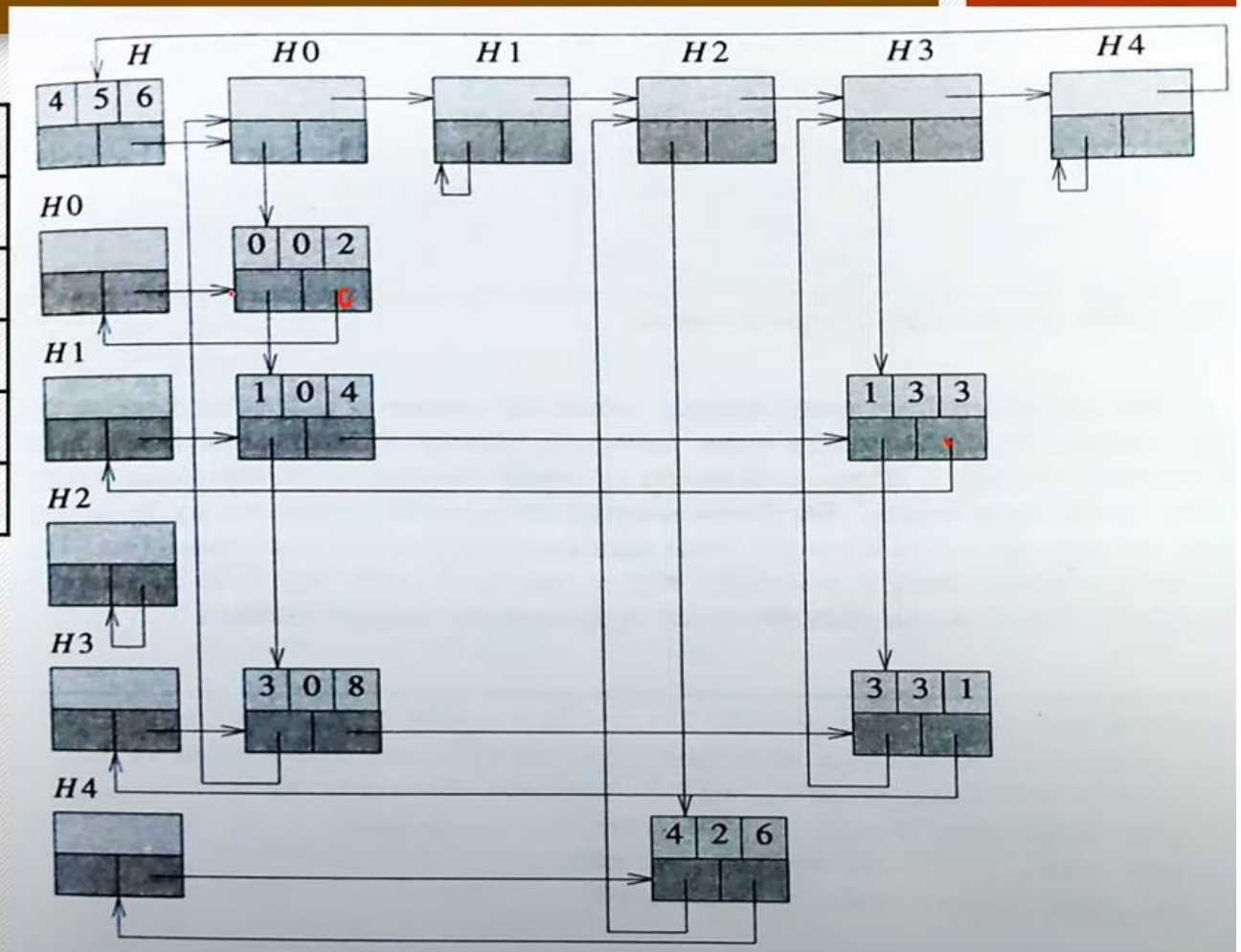
The elemental node has two pointers down, right and three other members: row, column and value (the row number and the column number where you have a non zero element)

Data structures and its applications

Linked list – Multi-list

	0	1	2	3
0	2	0	0	0
1	4	0	0	3
2	0	0	0	0
3	8	0	0	1
4	0	0	6	0

5X4



Data structures and its applications

Sparse matrix – node structure

```
typedef enum {head, entry} tagfield;
typedef struct {
    int row; int col; int value;
} entryNode;
struct MN {
        struct MN *down;
        struct MN *right;
        tagfield tag;
        union {
            struct MN *next;
            entryNode entry;
        } u;
};
```

THANK YOU

M S Anand

Department of Computer Science Engineering (AI&ML)

anandms@pes.edu